

Per-Job GPU Monitoring for Multi-Tenant Environments

LDMS User Group meeting
Feb 23, 2026

Nichamon Naksinehaboon
nichamon@ogc.us
Open Grid Computing

Node-level Metrics Miss Per-Job GPU Usage

Per-Node Metric

- GPU 0: 72% utilization
- When 3 jobs share a GPU:
Who owns that 72%?

Per-Job Metric

- Job 123 -> GPU 0: 72%
- Attribution requires:
GPU ID < – > Job ID correlation

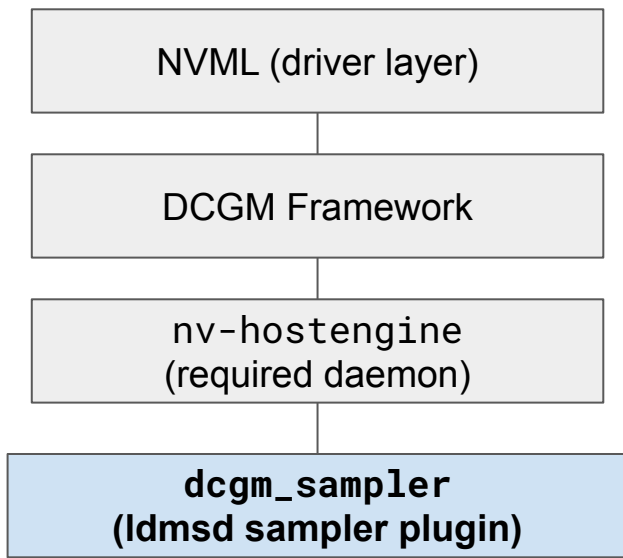
Outline and Scope

- What Idmsd provide today
- GPU allocation modes: exclusive and MIG
- MIG (Multi-instance GPU) – NVIDIA Ampere and later
- Slurm & Flux GPU scheduling with NVIDIA hardware

What Idmsd Have Today

dcgm_sampler

dcgm_sampler provides physical GPU metrics



Metrics collected (per physical GPU):

- Compute utilization %
- Memory used / free
- Temperature & power
- SM clock / memory bandwidth
- ECC errors / PCIe throughput

One metric set per physical GPU – indexed by integer (GPU 0, GPU 1, ...)

Question: GPU Vendors

What GPU vendor does your site use?

- Nvidia – All sites
- AMD – Some sites (LLNL, SNL, CSCS)
- Intel – None
- Other vendors – None

GPU Allocation on HPC Nodes

Two GPU Allocation Modes

These modes give each job a defined, isolated GPU resource.

- **Exclusive** – one job holds the entire GPU. No sharing. Standard default for HPC batch workloads
 - Hardware boundary between jobs
- **MIG (Multi-Instance GPU)** – Hardware partitioning of a physical GPU into isolated slices. Each slice has dedicated memory, compute, and cache. Introduced with Ampere (A100)
 - Hard hardware isolation between instances

Questions

- Which GPU scheduling modes does your site use?
 - Exclusive – Currently used at all sites
 - MIG – None, except LLNL that uses shared GPU on AMD GPU
 - Not sure – None
- Which scheduler do your end users use to allocate GPU?
 - Slurm – All
 - Flux – LLNL uses Flux + epilog to do shared GPU allocation
 - Others – None

Slurm

GPU request: `--gres=gpu:N` (sbatch/srun)

Environment variables set by Slurm

Variables	Value	Use for monitoring
CUDA_VISIBLE_DEVICES	0-based integer (non-MIG) <i>or</i> MIG UUID	GPU identity — mode-dependent
SLURM_JOB_GPUS	Physical GPU integer (global)	Reliable correlation to dcgm_sampler device ID
SLURM_STEP_GPUS	Physical GPU integer (global, per step)	Step-level correlation

CUDA_VISIBLE_DEVICES is remapped inside the **cgroup** to 0-based integers in non-MIG mode — does NOT match the global physical device ID.

MIG exception: in MIG mode, **CUDA_VISIBLE_DEVICES** contains the full instance UUID — not remapped.

Flux

GPU Request: `-g N`

What Flux sets:

- `CUDA_VISIBLE_DEVICES` – job-visible GPU identity
- Physical GPU index is available from the job's resource set (R object) via the Flux API

Flux supports exclusive allocation only.

Key Differences Between Slurm and Flux

	Slurm	Flux
Global physical GPU index	Direct SLURM_JOB_GPUS	R object via Flux API (physical index directly available)
MIG UUID	CUDA_VISIBLE_DEVICES	N/A (No MIG support)

Per-Job GPU Metrics By Scheduling Case

Exclusive Allocation – One job holds the GPU exclusively

- dcgm_sampler's device-level metrics are equivalent to per-job metrics when corresponding the GPU ID to a job
- Correlation with Slurm
 - SLURM_JOB_GPUS → physical GPU integer → dcgm_sampler device ID → direct match
 - Extend `slurm_notifier` to collect GPU devices allocated for jobs
- Correlation with Flux
 - R object (Flux API) → physical GPU integer → dcgm_sampler device ID → direct match

NOTE: Requires `ConstrainDevices=yes` in Slurm's `cgroup.conf`
This is to ensure that GPU device files are scoped to the job's cgroup.

MIG Isolation Exists (Slurm only)

What MIG gives you:

- Job gets its own GPU instance (GI) – dedicated memory, SM, cache
- No sharing with other jobs
- Each instance independently addressable in DCGM

What Slurm provides:

- `CUDA_VISIBLE_DEVICES` = MIG instance UUID
 - e.g., MIG-92b6c26a-... — globally unique, not remapped
- `SLURM_JOB_GPUS` = physical GPU integer only
 - Does *not* carry MIG UUID in MIG mode
- DCGM exposes per-instance metrics (`DCGM_FI_PROF_*` family) at the GPU instance entity level

NOTE: Requires `ConstrainDevices=yes` in Slurm's `cgroup.conf`
This is to ensure that GPU device files are scoped to the job's cgroup.

Values of CUDA_VISIBLE_DEVICES (With & Without MIG)

CUDA_VISIBLE_DEVICES behaves differently in exclusive and MIG modes

Exclusive (physical GPU)

- 0-based integer
- Remapped by cgroup → NOT global
- Job visibility only
- Use SLURM_JOB_GPUS for monitoring metrics correlation

MIG

- Full UUID string
- NOT remapped by cgroup → globally unique
- Correct anchor for dcgmGetGpuInstanceHierarchy() lookup
- Some older drivers used a different format: MIG-<GPU-UUID>/<GI-ID>/<CI-ID>

MIG Per-Job GPU Metrics: Possible Solution (Slurm Only)

Assemble things together:

- Slurm side
 - Capture MIG UUID from `CUDA_VISIBLE_DEVICES` at job start
 - A solution: Extend `slurm_notifier` to read `CUDA_VISIBLE_DEVICES`
- Idmsd side (a plugin)
 - Build UUID → GPU Instance Entity ID map via `dcgmGetGpuInstanceHierarchy()`
 - Query `DCGM_FI_PROF_*` metrics per GPU instance entity
 - Solutions: implement a new sampler plugin or expand current `dcgm_sampler`
- Attribution
 - Correlate Job ID → MIG UUID → GPU Instance Entity ID

Caveats: Power & temperature remain at physical GPU level

Caveat on Capturing MIG UUID

- `CUDA_VISIBLE_DEVICES` provides MIG UUID in the job's namespace
- `task_init_privileged` SPANK hook runs inside the job's namespace
 - `slurm_notifier` can read `CUDA_VISIBLE_DEVICES` at this step
- `nv-hostengine` the daemon `dcgm`

Question:

Does your site use MIG today?

- LLNL uses shared GPU on AMD GPUs and collect monitoring metrics at the physical device level
- No sites reported that NVIDIA MIG is in used but there are discussion to adopt it in near future

Summary of What's Possible

Allocation Type	Scheduler	Per-job Metrics?
Exclusive	Slurm/Flux	dcgm_sampler + job_ID & GPU device ID map
MIG, one job per instance	Slurm only	Possible solution exists

Q&A

- Chris added that `rdc_sampler` is a sampler plugin that collects AMD GPU metrics
- Tom asked if GPU slices can be configured after boot time
 - Chris: For AMD at LLNL, yes. Currently they let users requests what they want to do and process the request from the epilog
 - Steve: For NVIDIA at LANL, the number of slices can be changed at boot time.
- Tom's following questions when GPU UUID is generated (node boot time, job allocation time, etc)
 - This affects the values Idmsd needs to associate with job and GPU metrics
- With MiG, does each GPU instance/slice show up separately in the pci tree
 - No confirmation

Thank you!